# Student – Professor Metting Schedular

UCS 503 Software Engineering Project Report

Mid-Semester Evaluation

Submitted by:

(1024031015)Yogesh Goyal

(1024031011)Rakshit Chaudhary

(1024030759)Tanish Kumar Singla

BE Third Year, CoE/CSE

Group No:

Submitted to: Stuti Chugh



Computer Science and Engineering Department

TIET, Patiala

# TABLE OF CONTENTS


| S.No. | Assignment | Page No. |
|---|---|---|
| 1. | Project Selection Phase | |
| 1.1 | Software Bid | |
| 1.2 | Project Overview | |
| 2. | Analysis Phase | |
| 2.1 | Use Cases | |
| 2.1.1 | Use-Case Diagrams | |
| 2.1.2 | Use Case Templates | |
| 2.2 | Activity Diagram and Swimlane Diagrams | |
| 2.3 | Data Flow Diagrams (DFDs) | |
| 2.3.1 | DFD Level 0 | |
| 2.3.2 | DFD Level 1 | |
| 2.3.2 | DFD Level 2 | |
| 2.4 | Software Requirement Specification in IEEE Format | |
| 2.5 | User Stories and Story Cards | |

# 1. PROJECT SELECTION PHASE

## 1.1 Software Bid

### Project Title

Professor–Student Meeting Scheduler

### Project Tagline

An Automated Platform for Synchronized Academic Consultations, Availability Management, and Instant Notifications

### Problem Statement

Traditional scheduling of professor-student meetings is often handled through emails, messages, or informal communication. This creates several problems such as repeated back-and-forth communication, lack of real-time information about professor availability, double booking of time slots, missed cancellations, and additional administrative workload for faculty.

Students often do not have a reliable way to determine whether a professor is currently available for consultation. Similarly, professors have to manually manage meeting requests, availability, cancellations, and rescheduling.

The proposed **Professor–Student Meeting Scheduler** addresses these problems by providing a centralized platform where professors can manage their availability and students can view available slots and request meetings. The system also manages approvals, cancellations, rescheduling, and automatic notifications.

The PPT specifically identifies the major existing challenges as **uncoordinated email threads, no real-time visibility, frequent schedule conflicts, and administrative overhead**.

### Proposed Solution

The proposed system is an automated academic appointment-management platform that synchronizes student meeting requests with professor availability.

Professors can create and manage recurring consultation slots, ad-hoc availability, and blackout periods. Students can browse available slots and submit meeting requests. The system checks availability and prevents overlapping appointments.

After a request is submitted, the professor can accept, reject, or request rescheduling. The system automatically updates the meeting status and sends notifications to the relevant users.

The system therefore provides:

- Dynamic professor availability management
- Self-service student booking
- Automatic conflict checking

- Meeting approval and rejection
- Meeting rescheduling
- Meeting cancellation
- Automated notifications
- Centralized meeting tracking
- Profile and meeting history management

## Objectives

The main objectives of the project are:

1. To provide students with real-time information about professor availability.
2. To allow professors to create and manage their consultation schedules.
3. To allow students to independently request meetings.
4. To prevent double bookings and scheduling conflicts.
5. To allow professors to accept, reject, or reschedule meeting requests.
6. To automatically notify students and professors about meeting-related events.
7. To provide centralized tracking of meeting statuses and history.
8. To reduce manual communication and administrative workload.
9. To provide a responsive interface accessible through desktops, tablets, and smartphones.
10. To maintain secure role-based access for students and professors.

## SDLC Model Selection

## Selected Model: Agile Scrum

Agile Scrum has been selected for the development of the Professor–Student Meeting Scheduler because the project requirements can evolve based on feedback from students and professors.

The system can be divided into multiple independent features such as authentication, availability management, meeting booking, approval management, notifications, and reporting. These features can be developed incrementally through short sprints.

## Justification for Agile Scrum

| Criteria | Agile Scrum |
|---|---|
| Requirement Adaptability | Requirements can be refined during every sprint. |
| Feedback | Continuous feedback can be obtained from students and professors. |
| Risk Management | Bugs and risks can be identified early. |
| Testing | Testing can be performed continuously. |
| Delivery | Features can be delivered incrementally. |
| MVP | Authentication and Booking can be delivered first. |

Therefore, Agile Scrum is suitable because it supports continuous improvement, frequent feedback, early testing, and incremental delivery.

## 1.2 Project Overview

## Introduction

The **Professor–Student Meeting Scheduler** is an automated platform designed to simplify and organize academic consultations between students and professors.

The system replaces manual appointment scheduling with a centralized digital platform. Students can view professor availability and request meetings, while professors can manage their availability and process meeting requests.

The system also provides automated notifications whenever a meeting is requested, accepted, rejected, rescheduled, or cancelled.

## Existing System

In the existing manual system, students generally communicate with professors through emails or messages to request meetings. This can result in:

- Multiple back-and-forth messages
- Delayed responses
- Unclear professor availability
- Double bookings
- Missed cancellations
- Manual schedule tracking

## Proposed System

The proposed system provides:

- Online authentication
- Professor availability management
- Student meeting requests
- Availability and conflict checking
- Meeting approval/rejection
- Meeting rescheduling
- Meeting cancellation
- Automated notifications
- Meeting history
- Profile management
- Administrative reporting and configuration in the expanded model

## Main Actors

## Student

The student can:

- Register/Login
- View profile
- View professor availability
- Request a meeting
- View meeting status
- Cancel meetings
- Receive notifications

## Professor

The professor can:

- Login
- View/update profile
- Set availability
- View meeting requests
- Accept/reject requests
- Reschedule meetings
- View meeting history
- Receive notifications

## Administrator

The expanded use-case model includes administrative functions such as:

- System configuration
- System report generation
- Profile management
- Notification-related administration

## Payment Gateway

The expanded use-case model also represents an optional **Payment Gateway** for processing meeting fees.

# 2. ANALYSIS PHASE

## 2.1 Use Cases

A use case describes a specific interaction between an actor and the system to achieve a particular goal.

The main actors of the Professor–Student Meeting Scheduler are **Student, Professor, Administrator, and an optional Payment Gateway**.

### Student Use Cases

1. Register/Login
2. View Profile
3. View Professor Availability
4. Request Meeting
5. View Meeting Status
6. Cancel Meeting
7. Receive Notifications

### Professor Use Cases

1. Set Availability
2. View Meeting Requests
3. Accept/Reject Request
4. Reschedule Meeting
5. View History
6. Receive Notifications
7. View/Update Profile

### Administrator Use Cases

1. View Profile
2. Receive Notifications
3. Generate System Reports
4. System Configuration

## Optional Payment Use Case

## Process Meeting Fees

This interacts with the external Payment Gateway shown in the expanded use-case model.

---

## 2.1.1 Use-Case Diagram

## Use-Case Diagram: Professor–Student Meeting Scheduler

The use-case diagram represents the functional interaction between the external actors and the Professor–Student Meeting Scheduler system.

The Student actor interacts with the system to register/login, view profile, view professor availability, request meetings, view meeting status, cancel meetings, and receive notifications.

The Professor actor interacts with the system to manage availability, view meeting requests, accept or reject requests, reschedule meetings, view meeting history, manage profile information, and receive notifications.

The expanded model also represents an Administrator responsible for system configuration and report generation. An optional Payment Gateway is represented for processing meeting fees.

**Important relationships**

**Register/Login → <<include>> Authenticate User**

Authentication is required whenever a user logs into the system.

**Request Meeting → <<include>> Check Availability**

Before creating a meeting request, the system must verify that the selected slot is available.

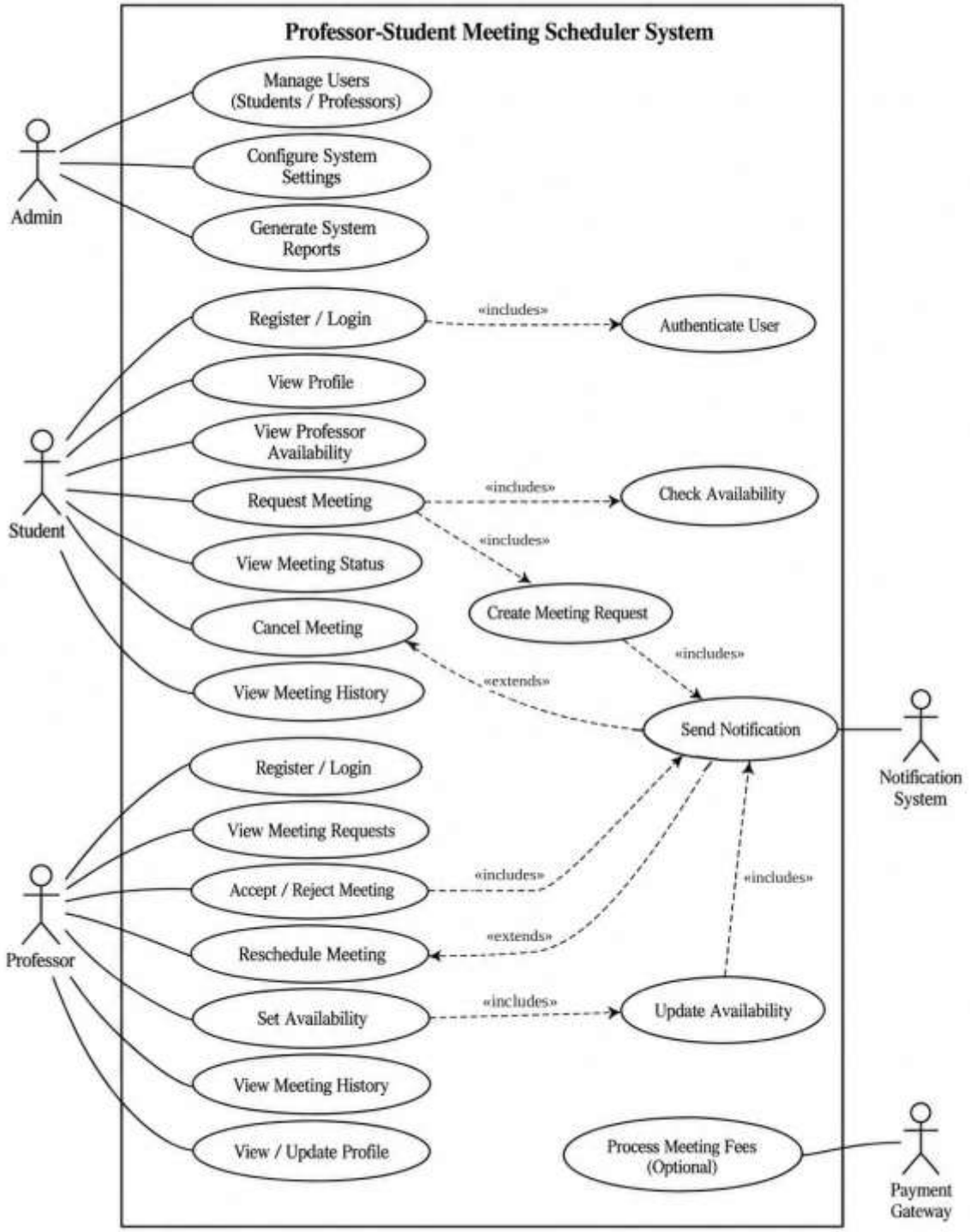
**Meeting Operations → <<include>> Send Notification**

Notifications are generated during important meeting lifecycle events such as booking, acceptance, rejection, rescheduling, and cancellation.

**Reset Password → <<extend>> Register/Login**

Password reset represents optional behavior that can occur when a user has forgotten their credentials.

## Use Case Diagram – Professor Student Meeting Scheduler



### 2.1.2 Use Case Templates

---

## UC-01: Register/Login

**Actor:** Student, Professor

**Description:**
Allows a student or professor to register or authenticate into the system.

**Preconditions:**

- User must have valid registration/login information.
- System must be available.

**Main Flow:**

1. User opens the application.
2. User enters login or registration details.
3. System validates the information.
4. System authenticates the user.
5. System creates an authenticated session.
6. User is redirected to the appropriate dashboard.

**Alternative Flow:**
If the credentials are invalid, the system displays an error message and authentication fails.

**Postcondition:**
User is successfully authenticated and can access authorized functions.

---

## UC-02: View Professor Availability

**Actor:** Student

**Description:**
Allows students to view available consultation slots of a professor.

**Preconditions:**

- Student is authenticated.
- Professor information exists.

**Main Flow:**

1. Student selects a professor.
2. Student requests availability.
3. System retrieves the professor's availability.
4. System displays available slots.

**Alternative Flow:**
If no slots are available, the system displays an appropriate message.

**Postcondition:**
Student can select an available consultation slot.

---

## UC-03: Request Meeting

**Actor:** Student

**Description:**
Allows a student to request a meeting with a professor.

**Preconditions:**

- Student is authenticated.
- Professor has available slots.

**Main Flow:**

1. Student selects professor.
2. Student selects an available slot.
3. System checks slot availability.
4. System checks for conflicts.
5. System creates a meeting request.
6. Meeting is stored as pending.
7. Notification is generated.

**Alternative Flow:**
If the selected slot is no longer available, the request is rejected.

**Postcondition:**
A pending meeting request is created.

---

## UC-04: Accept/Reject Meeting Request

**Actor:** Professor

**Description:**
Allows a professor to accept or reject a student's meeting request.

**Preconditions:**

- Professor is authenticated.
- A pending meeting request exists.

**Main Flow:**

1. Professor opens meeting requests.
2. Professor selects a pending request.
3. Professor chooses Accept or Reject.
4. System updates the meeting status.
5. System generates a notification.

**Alternative Flow:**
Professor may request rescheduling instead of accepting/rejecting directly.

**Postcondition:**
Meeting status is updated and the student is notified.

---

## UC-05: Set Availability

**Actor:** Professor

**Description:**
Allows professors to create and modify consultation availability.

**Preconditions:**

- Professor is authenticated.

**Main Flow:**

1. Professor opens availability management.
2. Professor enters consultation hours.
3. Professor selects recurring or ad-hoc availability.
4. Professor can define blackout periods.
5. System validates the time information.
6. System stores the availability.

**Alternative Flow:**
Invalid or conflicting time information is rejected.

**Postcondition:**
Updated professor availability is stored.

---

## UC-06: Reschedule Meeting

**Actor:** Professor

**Description:**
Allows a professor to change the time of an existing meeting.

**Preconditions:**

- Existing meeting is available for rescheduling.
- Professor is authorized.

**Main Flow:**

1. Professor selects the meeting.
2. Professor proposes a new time.
3. System checks availability.
4. System updates the meeting.
5. System generates a rescheduling notification.

**Alternative Flow:**
If the new slot is unavailable, rescheduling is rejected.

**Postcondition:**
Meeting is assigned the new valid time.

---

## UC-07: Cancel Meeting

**Actor:** Student

**Description:**
Allows a student to cancel an existing meeting.

**Preconditions:**

- Student is authenticated.
- Meeting exists and is eligible for cancellation.

**Main Flow:**

1. Student selects the meeting.
2. Student selects Cancel.
3. System updates meeting status.
4. System sends notification.

**Alternative Flow:**
If cancellation is not permitted, the system displays an error.

**Postcondition:**
Meeting status becomes Cancelled.

---

## UC-08: View Meeting Status

**Actor:** Student

**Description:**
Allows students to view the current status of their meetings.

**Main Flow:**

1. Student opens meeting dashboard.
2. System retrieves meeting records.
3. System displays current status.

Possible statuses include:

- Pending
- Accepted
- Rejected
- Rescheduled
- Cancelled

**Postcondition:**
Student can view the latest status of meetings.

---

## UC-09: Receive Notifications

**Actor:** Student, Professor

**Description:**
Provides automatic notifications for important meeting events.

**Trigger Events:**

- Meeting request submitted
- Meeting accepted
- Meeting rejected
- Meeting rescheduled
- Meeting cancelled

**Postcondition:**
Relevant users receive notifications and notification records are maintained.

---

## UC-10: Generate System Reports

**Actor:** Administrator

**Description:**
Allows the administrator to generate reports related to system activity.

**Precondition:**
Administrator is authenticated and authorized.

**Main Flow:**

1. Administrator opens reporting section.
2. Selects report type.
3. System retrieves required information.
4. System generates report.

**Postcondition:**
Requested report is generated.

---

## UC-11: System Configuration

**Actor:** Administrator

**Description:**
Allows the administrator to manage supported system-level configuration.

**Precondition:**
Administrator is authenticated.

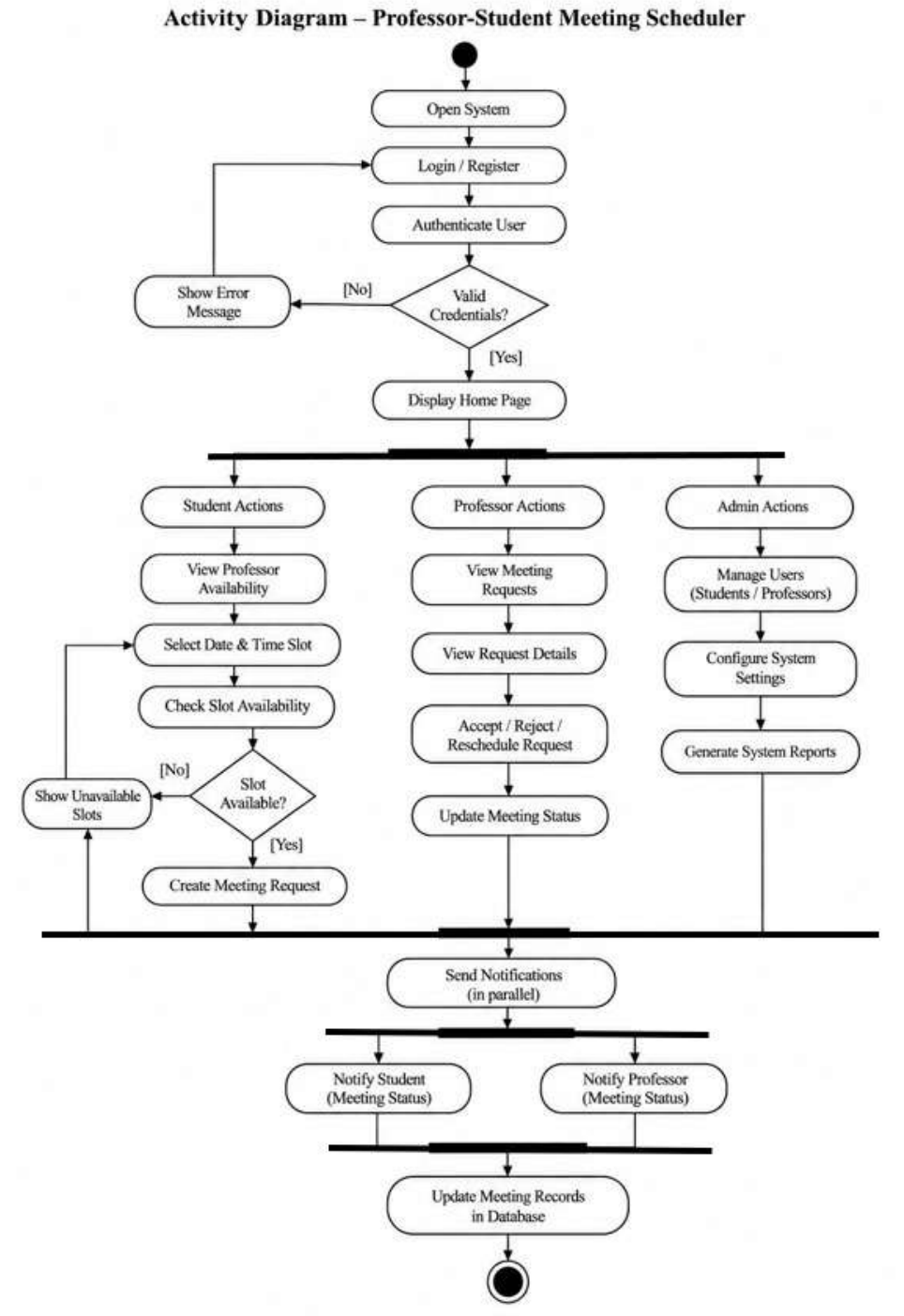
**Main Flow:**

1. Administrator opens configuration.
2. Modifies required settings.
3. System validates the configuration.
4. System stores the changes.

**Postcondition:**
System configuration is updated.

---

## 2.2 Activity Diagram and Swimlane Diagrams



The activity diagram represents the complete workflow of scheduling a meeting between a student and a professor.

The process begins when the student opens the application and logs in or registers. The system authenticates the user. If authentication fails, an error message is displayed. If authentication succeeds, the student accesses the dashboard.

The student then selects a professor and views the professor's available consultation slots. The system retrieves the availability information and displays it to the student.

The student selects a suitable slot and submits a meeting request. The system checks whether the selected slot is still available and whether any scheduling conflict exists.

If the slot is unavailable, the system informs the student. If the slot is available, a meeting request is created and stored.

The professor receives the meeting request and can accept, reject, or request rescheduling. The system updates the meeting status accordingly.

Finally, the notification system sends the relevant notification to the student and professor.

## Swimlane Diagram

You can divide the diagram into four lanes:

**Student | System | Professor | Notification System**

## Student activities

- Register/Login
- Select Professor
- View Availability
- Select Slot
- Request Meeting
- View Status
- Receive Notification

## System activities

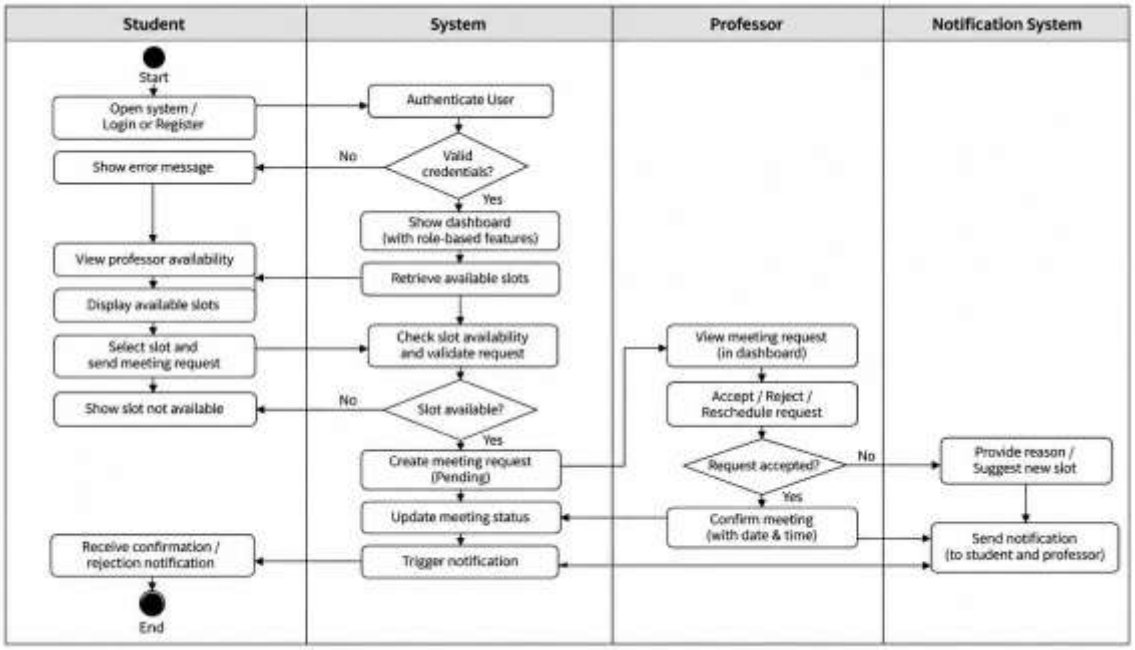
- Authenticate User
- Retrieve Availability
- Check Availability
- Create Meeting
- Update Meeting Status

## Professor activities

- View Meeting Request
- Accept Request
- Reject Request
- Request Rescheduling
- Manage Availability

## Notification System

- Generate Notification
- Send Notification
- Record Notification



## 2.3 Data Flow Diagrams (DFDs)

### Introduction

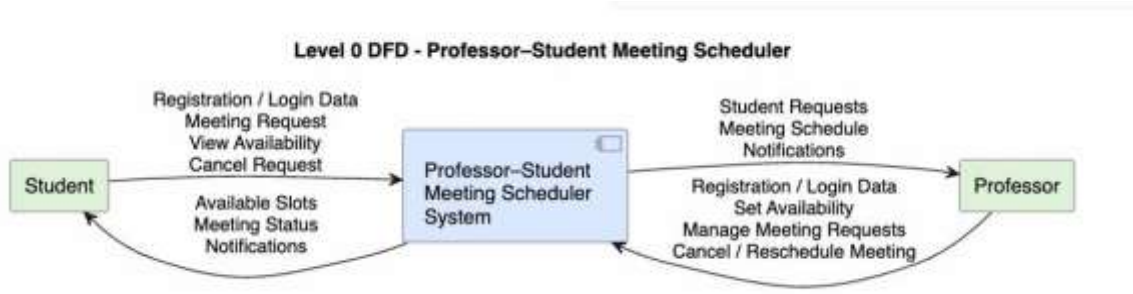
Data Flow Diagrams represent how information enters, moves through, and leaves the Professor–Student Meeting Scheduler.

The system is represented at three levels:

- Level 0 – Overall system context
- Level 1 – Major system processes
- Level 2 – Detailed process decomposition

---

### 2.3.1 DFD Level 0




DFD Level 0 represents the Professor–Student Meeting Scheduler as a single process.

The two primary external entities are:

- Student
- Professor

The internal databases and detailed processes are hidden at this level.

### Student → System

The Student provides:

- Registration/Login information
- Search criteria
- Slot inquiry
- Meeting request
- Cancellation information

### System → Student

The System provides:

- Professor availability
- Booking confirmation
- Meeting status
- Cancellation acknowledgement
- Notifications

### Professor → System

The Professor provides:

- Login credentials
- Availability schedules
- Accept/Reject decisions
- Rescheduling information

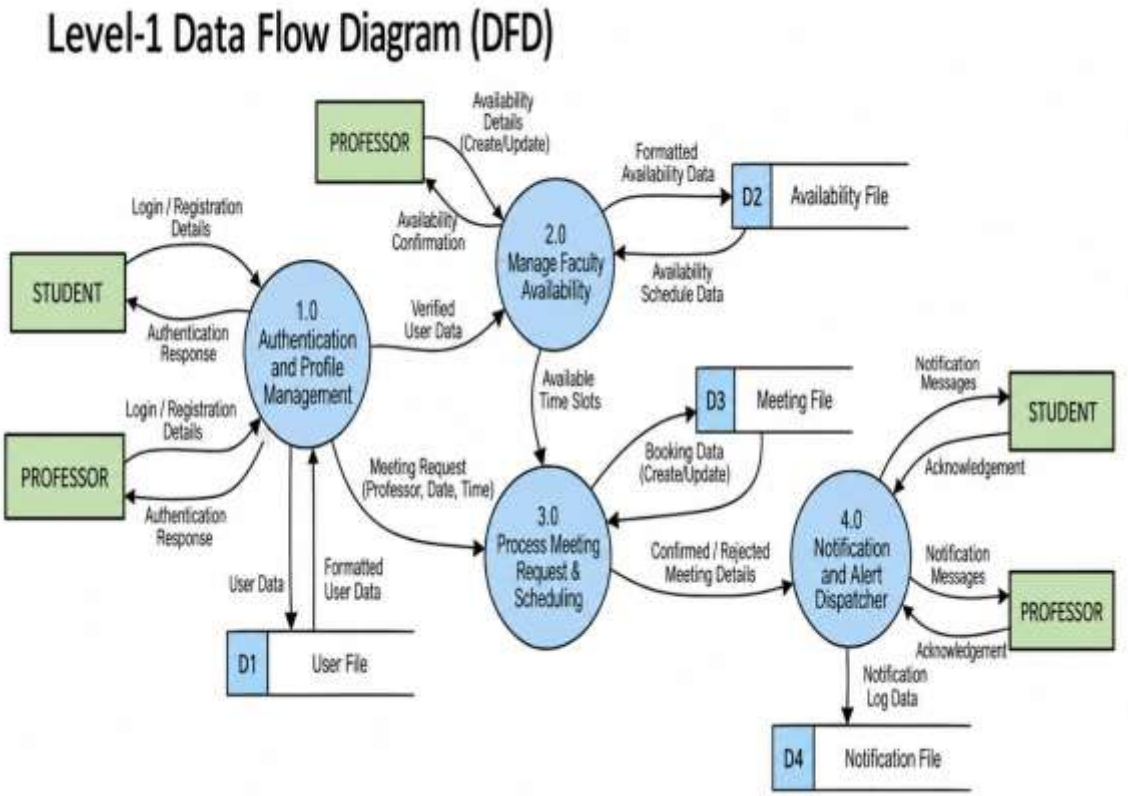
### System → Professor

The System provides:

- Pending meeting requests
- Appointment agenda
- Schedule information
- Notifications
- Meeting status

---

### 2.3.2 DFD Level 1




At Level 1, the system is decomposed into four major processes.

## Process 1.0 – User Authentication & Profile Management

This process handles:

- User registration
- Login
- Credential validation
- Session/token creation
- Profile synchronization

**Data Store:** D1 – User Store

---

## Process 2.0 – Faculty Availability Management

This process handles:

- Recurring availability
- Ad-hoc availability
- Blackout periods
- Time validation
- Availability updates

**Data Store:** D2 – Availability Store

---

## Process 3.0 – Meeting Request & Scheduling Engine

This process:

- Receives meeting requests
- Checks availability
- Checks conflicts
- Creates pending meetings
- Updates meeting status
- Stores confirmed meetings

**Data Store:** D3 – Meeting Records

---

## Process 4.0 – Notification & Alert Dispatcher

This process monitors meeting state changes such as:

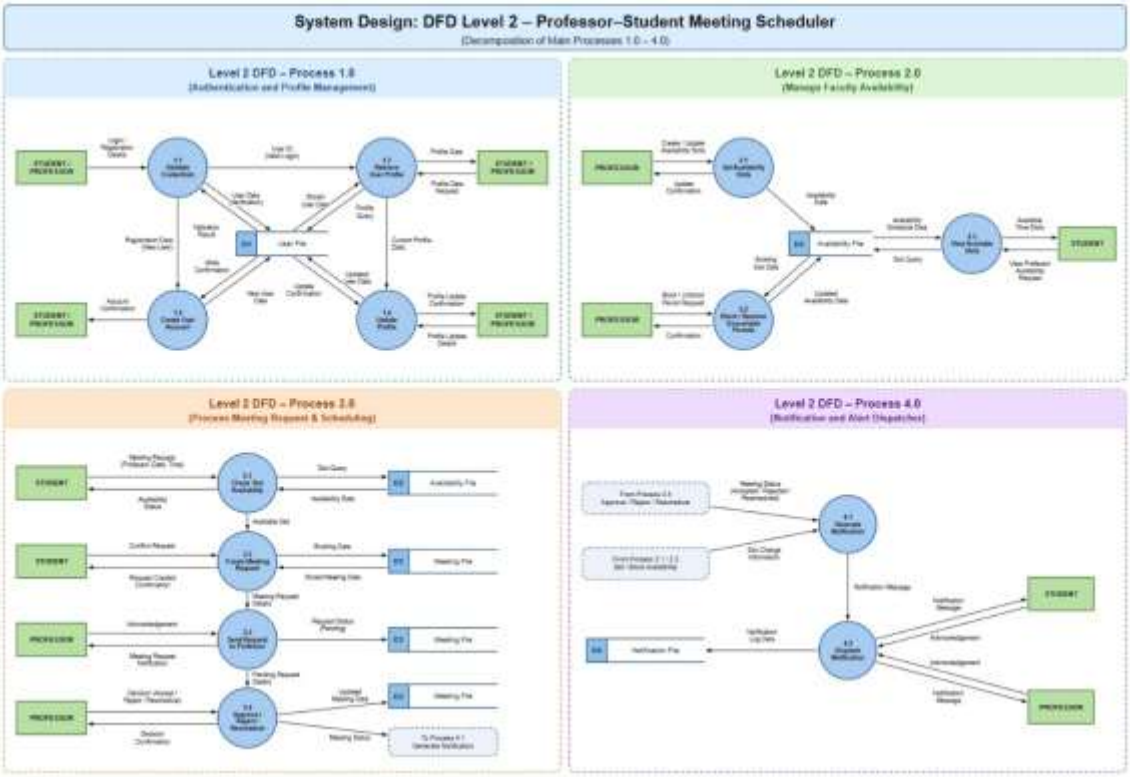
- Created
- Accepted
- Rejected
- Rescheduled
- Cancelled

It then creates and sends appropriate notifications.

**Data Store:** D4 – Notification Store

---

## 2.3.3 DFD Level 2




DFD Level 2 further decomposes the major processes into smaller activities.

## Process 1.0 – User Authentication

### 1.1 Validate Credentials

The system receives username/email and password and validates the credentials.

### 1.2 Create Session

After successful authentication, the system creates an authenticated session/token.

## Data Store

### D1 – User Store

---

## Process 2.0 – Meeting Management

### 2.1 Check Availability

The system checks whether the selected professor slot is available.

### 2.2 Create Meeting

After successful availability validation, the system creates the meeting request.

### 2.3 Send Notification

The system sends a notification to the relevant professor/student.

**Data Stores**

- D2 – Meeting Database
- D3 – Availability Database

---

## Process 3.0 – Availability Management

### 3.1 Validate Time Slots

The system validates professor-provided availability information.

### 3.2 Update Availability

The system stores the updated availability information.

**Data Store**

**D3 – Availability Database**

---

## Process 4.0 – Notification Management

### 4.1 Generate Notification

The system receives an event such as meeting creation, acceptance, rejection, rescheduling, or cancellation.

It generates the appropriate notification and records it.

**Data Store**

**D4 – Notification Log**

---

## 2.4 Software Requirement Specification in IEEE Format

### 2.4.1 Introduction

### Purpose

The purpose of this Software Requirements Specification is to define the functional and non-functional requirements of the Professor–Student Meeting Scheduler.

The system is designed to automate the scheduling of academic consultations between students and professors and to provide centralized availability, booking, meeting management, and notification services.

### Scope

The system provides:

- User authentication
- Role-based access
- Professor availability management
- Student meeting requests
- Availability checking
- Conflict prevention
- Meeting approval/rejection
- Meeting rescheduling
- Meeting cancellation
- Automated notifications
- Meeting history
- Profile management
- Administrative functions

The system is intended to reduce manual communication and improve the efficiency of academic consultation scheduling.

---

## 2.4.2 Overall Description

## Product Perspective

The Professor–Student Meeting Scheduler is a centralized software system that connects students and professors through an automated appointment-management platform.

The system contains:

- User authentication
- Profile management
- Availability management
- Meeting management
- Notification management
- Administrative functions

## User Classes

**Student**

Uses the system to search professor availability and request/manage meetings.

**Professor**

Uses the system to publish availability and process student meeting requests.

**Administrator**

Manages system-level configuration and generates reports.

---

### 2.4.3 Functional Requirements

#### FR-01: User Authentication

The system shall provide secure role-based login and session authentication for students and professors.

#### FR-02: Availability Scheduling

The system shall allow professors to define recurring consultation slots, ad-hoc availability, and blackout periods.

#### FR-03: Availability Viewing

The system shall allow students to view current professor availability.

#### FR-04: Booking and Conflict Checking

The system shall immediately validate meeting requests against overlapping appointments.

#### FR-05: Approval Management

The system shall allow professors to accept, reject, or request rescheduling of meeting requests.

#### FR-06: Meeting Cancellation

The system shall allow eligible meetings to be cancelled.

#### FR-07: Meeting Status

The system shall allow users to view the current status of meetings.

#### FR-08: Profile Management

The system shall allow users to view and update their profiles.

#### FR-09: Automated Notifications

The system shall send notifications when meetings are requested, accepted, rejected, rescheduled, or cancelled.

#### FR-10: Administrative Reporting

The system shall allow authorized administrators to generate system reports.

#### FR-11: System Configuration

The system shall allow authorized administrators to manage supported system configuration.

#### FR-12: Optional Payment Processing

The system may interact with an external Payment Gateway for the optional meeting-fee process shown in the use-case model.

---

### 2.4.4 Non-Functional Requirements

## Performance

The system should provide **sub-second response time** for availability browsing and booking confirmations.

## Security

- Passwords shall be encrypted using Bcrypt.
- JWT shall be used for session authentication.
- Role-based authorization shall restrict access to protected functions.

## Availability and Reliability

The system targets **99.5% uptime** during operational office and examination weeks.

## Usability

The interface shall be responsive and accessible through:

- Desktop computers
- Tablets
- Smartphones

## Data Integrity

The system shall maintain ACID-compliant transactions to prevent:

- Phantom bookings
- Duplicate confirmations
- Inconsistent meeting states

---

### 2.4.5 External Interface Requirements

#### User Interface

The system shall provide a responsive web interface for students, professors, and administrators.

#### Software Interface

The frontend shall communicate with backend services through defined API interfaces.

#### Database Interface

The system shall use a relational database for storing:

- User information
- Professor availability
- Meeting records
- Notification records

#### Authentication Interface

The system shall use secure authentication mechanisms including Bcrypt password encryption and JWT-based sessions.

#### Notification Interface

The notification component shall communicate meeting lifecycle events to relevant users.

#### Payment Interface

The optional Payment Gateway shall be used for processing meeting fees when applicable.

---

### 2.4.6 Constraints

The major system constraints are:

1. The system must maintain secure role-based access.
2. Meeting bookings must be validated for conflicts.
3. Database transactions must preserve data integrity.
4. The system must provide responsive access across supported devices.
5. Authentication must use secure mechanisms.
6. The system must maintain reliable availability during important academic periods.

---

### 2.4.7 Feasibility Study

## Technical Feasibility

The project is technically feasible because it can be developed using commonly available technologies and standard web-development skills.

The proposed technology stack includes:

- React
- Node.js
- PostgreSQL
- Bcrypt
- JWT

No specialized hardware is required.

## Operational Feasibility

The system is operationally feasible because it simplifies the existing scheduling process for both students and professors.

Students can independently check availability and request appointments, while professors can manage their schedules through the platform.

## Economic Feasibility

The system has relatively low infrastructure requirements because it can be deployed using standard cloud/PaaS infrastructure.

The benefits include:

- Reduced administrative work
- Reduced scheduling errors
- Reduced communication overhead
- Better utilization of professor consultation time

## Schedule Feasibility

The project can be developed incrementally using Agile Scrum and two-week sprints. Core features such as authentication and booking can be developed first, followed by availability management, notifications, administration, and other enhancements.

---

## 2.5 User Stories and Story Cards

### User Story 1 – View Professor Availability

**As a Student, I want to view a professor's available consultation slots so that I can select a suitable meeting time.**

**Acceptance Criteria**

- Student can select a professor.
- System displays available slots.
- Unavailable slots cannot be selected.

---

### User Story 2 – Request Meeting

**As a Student, I want to request a meeting with a professor so that I can schedule an academic consultation.**

**Acceptance Criteria**

- Student selects an available slot.
- System checks availability.
- System checks conflicts.
- Meeting request is created.
- Professor receives notification.

---

### User Story 3 – View Meeting Status

**As a Student, I want to view my meeting status so that I know whether my request has been accepted, rejected, rescheduled, or cancelled.**

**Acceptance Criteria**

- Student can view meeting history.
- Latest meeting status is displayed.
- Status is updated whenever the professor takes action.

---

### User Story 4 – Cancel Meeting

**As a Student, I want to cancel an eligible meeting so that the professor is informed about the cancellation.**

**Acceptance Criteria**

- Student can select an eligible meeting.
- Meeting status changes to cancelled.
- Professor receives a notification.

---

## User Story 5 – Manage Availability

**As a Professor, I want to define my consultation availability so that students can see when I am available.**

**Acceptance Criteria**

- Professor can create recurring slots.
- Professor can create ad-hoc slots.
- Professor can define blackout periods.
- Availability is stored successfully.

---

## User Story 6 – Process Meeting Requests

**As a Professor, I want to accept or reject meeting requests so that I can manage my consultation schedule.**

**Acceptance Criteria**

- Professor can view pending requests.
- Professor can accept a request.
- Professor can reject a request.
- Student receives the updated status.

---

## User Story 7 – Reschedule Meeting

**As a Professor, I want to reschedule a meeting so that I can handle changes in my availability.**

**Acceptance Criteria**

- Professor can propose a new time.
- System checks the new slot.
- Meeting is updated.
- Student receives a notification.

---

## User Story 8 – Receive Notifications

**As a Student or Professor, I want to receive notifications about meeting changes so that I remain informed about the appointment lifecycle.**

**Acceptance Criteria**

Notifications are generated for:

- New meeting request
- Acceptance
- Rejection
- Rescheduling
- Cancellation

---

## User Story 9 – View Meeting History

**As a Professor, I want to view my meeting history so that I can track previous consultations.**

**Acceptance Criteria**

- Previous meetings are stored.
- Professor can view relevant meeting records.
- Meeting status and schedule are displayed.

---

## User Story 10 – Generate Reports

**As an Administrator, I want to generate system reports so that I can monitor system activity.**

**Acceptance Criteria**

- Administrator can access reporting.
- Relevant system records can be retrieved.
- Report is generated successfully.

---

## Story Cards

For each story card, use this format:

## Story Card 1 – View Professor Availability

**Story:**
As a Student, I want to view professor availability so that I can select a suitable consultation slot.

**Priority:** High

**Actor:** Student

**Precondition:** Student is logged in.

**Acceptance Criteria:** Available professor slots are displayed and unavailable slots cannot be booked.

---

## Story Card 2 – Request Meeting

**Story:**
As a Student, I want to request a meeting so that I can consult a professor.

**Priority:** High

**Actor:** Student

**Precondition:** Student is authenticated and a valid slot exists.

**Acceptance Criteria:** Availability is checked, conflicts are prevented, and a meeting request is created.

---

## Story Card 3 – Manage Availability

**Story:**
As a Professor, I want to manage my availability so that students can see my consultation hours.

**Priority:** High

**Actor:** Professor

**Precondition:** Professor is authenticated.

**Acceptance Criteria:** Recurring, ad-hoc, and blackout periods can be stored.

---

## Story Card 4 – Approve/Reject Meeting

**Story:**
As a Professor, I want to accept or reject meeting requests so that I can control my consultation schedule.

**Priority:** High

**Actor:** Professor

**Precondition:** A pending meeting request exists.

**Acceptance Criteria:** Meeting status is updated and the student is notified.

---

## Story Card 5 – Reschedule Meeting

**Story:**
As a Professor, I want to reschedule a meeting so that schedule changes can be handled.

**Priority:** Medium

**Actor:** Professor

**Precondition:** Existing meeting is available for rescheduling.

**Acceptance Criteria:** New slot is checked for availability and the updated meeting details are stored.

---

## Story Card 6 – Notifications

**Story:**
As a Student/Professor, I want to receive meeting notifications so that I know when meeting information changes.

**Priority:** High

**Actor:** Student / Professor

**Precondition:** A meeting lifecycle event occurs.

**Acceptance Criteria:** Appropriate notification is generated and delivered.

---

## Story Card 7 – Cancel Meeting

**Story:**
As a Student, I want to cancel a meeting so that the professor is informed.

**Priority:** Medium

**Actor:** Student

**Precondition:** Student has an eligible meeting.

**Acceptance Criteria:** Meeting becomes cancelled and notification is generated.

---

## Story Card 8 – Generate System Reports

**Story:**
As an Administrator, I want to generate system reports so that I can monitor system activity.

**Priority:** Medium

**Actor:** Administrator

**Precondition:** Administrator is authenticated.

**Acceptance Criteria:** Required report is generated from available system records.

---